

SECURITY ASSESSMENT REPORT

OWASP Juice Shop — Attack Surface Mapping

Task1 — Reconnaissance & Attack Surface Discovery

Field	Detail
Target Application	OWASP Juice Shop
Target IP Address	192.168.52.1
Target Port	3000/TCP
Base URL	http://192.168.52.1:3000
Assessment Client	Kali Linux
Assessment Type	Authorized laboratory assessment
Report Date	October 1, 2026

CONFIDENTIAL — For educational / laboratory use only

1. Executive Summary

This report documents a security assessment performed against an intentionally vulnerable OWASP Juice Shop application hosted in a controlled laboratory environment at <http://192.168.52.1:3000>. The assessment was conducted from a Kali Linux client as an authorized vulnerability assessment and penetration-testing exercise.

The assessment began with network and service enumeration, followed by web application reconnaissance, directory and file enumeration, technology fingerprinting, and application API analysis. Testing confirmed that the Juice Shop application was accessible over TCP port 3000, and web technology identification confirmed the target as OWASP Juice Shop.

Further enumeration identified a hidden `/ftp/` directory despite it being explicitly disallowed in `robots.txt`. The directory listing was found to be enabled and exposed multiple files, including confidential-looking documents, backup files, encrypted material, configuration-related files, and a KeePass credential database.

The assessment successfully demonstrated the Directory Listing / Confidential Document challenge, which Juice Shop's own Challenge API subsequently confirmed as solved (`directoryListingChallenge`). Additional observations included an accessible product search API, an enumerable internal Challenge API describing 116 possible security challenges, and server-side restrictions on certain file extensions that nonetheless leaked an internal source-code path.

No claim is made that every identified Juice Shop challenge was successfully exploited. This report therefore separates confirmed findings from identified attack surface that warrants further testing.

2. Scope

The assessment was limited to the following laboratory target:

Item	Value
Target IP	192.168.52.1
Target Port	3000/TCP
Application	OWASP Juice Shop
Base URL	http://192.168.52.1:3000
Client	Kali Linux
Assessment Type	Authorized laboratory assessment

3. Objectives

- Identify exposed network services.
- Identify the web application and technology stack.
- Enumerate accessible web directories and files.

- Identify sensitive or improperly exposed resources.
- Investigate application endpoints.
- Test whether discovered resources could be accessed without authorization.
- Document successful vulnerabilities and supporting evidence.
- Identify additional attack surfaces for potential further testing.

4. Methodology

The assessment followed a reconnaissance-to-validation methodology across eight phases:

Phase	Activity
1	Network Enumeration — identify exposed services with Nmap
2	Web Technology Identification — fingerprint the stack with WhatWeb
3	Web Directory Enumeration — enumerate common paths with Gobuster
4	Robots.txt Analysis — inspect disallowed paths for information disclosure
5	FTP/Web File Enumeration — access and enumerate the discovered /ftp/ directory
6	File Analysis — request and examine individually exposed documents
7	Application API Enumeration — query the challenge API and product search endpoint
8	Challenge Validation — confirm which Juice Shop challenges were actually completed

5. Reconnaissance & Findings

5.1 Network Enumeration

The target was scanned using Nmap to identify exposed services:

```
nmap -sV -p 3000 192.168.52.1
```

The scan identified TCP port 3000 as open, running the Juice Shop web application. No other exposed services were identified within the tested scope.

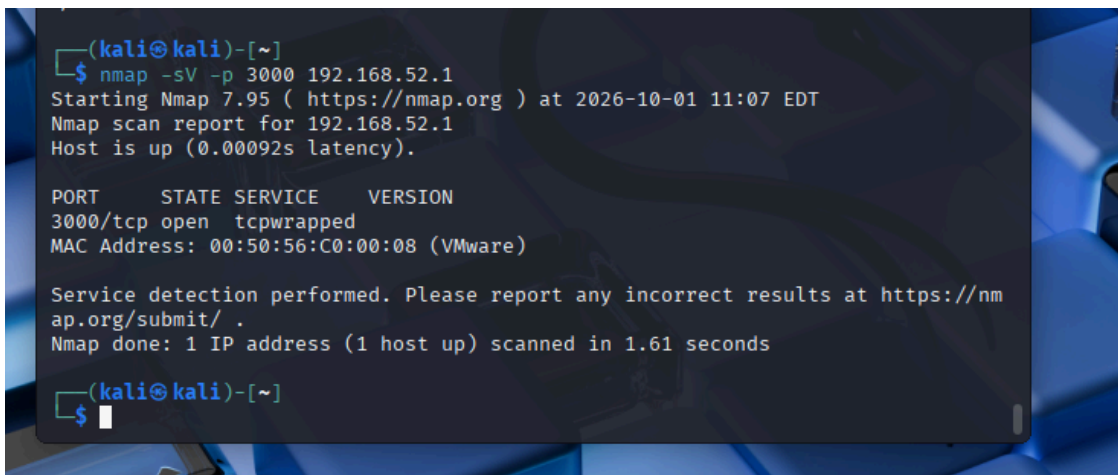


Figure 1 — Nmap service scan confirming TCP/3000 open

Finding: TCP/3000 — HTTP Web Application

The target exposed a web application directly on port 3000, with no evidence of a reverse proxy obscuring the service banner.

5.2 Web Technology Identification

The application was fingerprinted using WhatWeb:

```
whatweb http://192.168.52.1:3000
```

The response identified the application as OWASP Juice Shop, an HTML5/JavaScript single-page application. Notable response headers included:

```
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
```

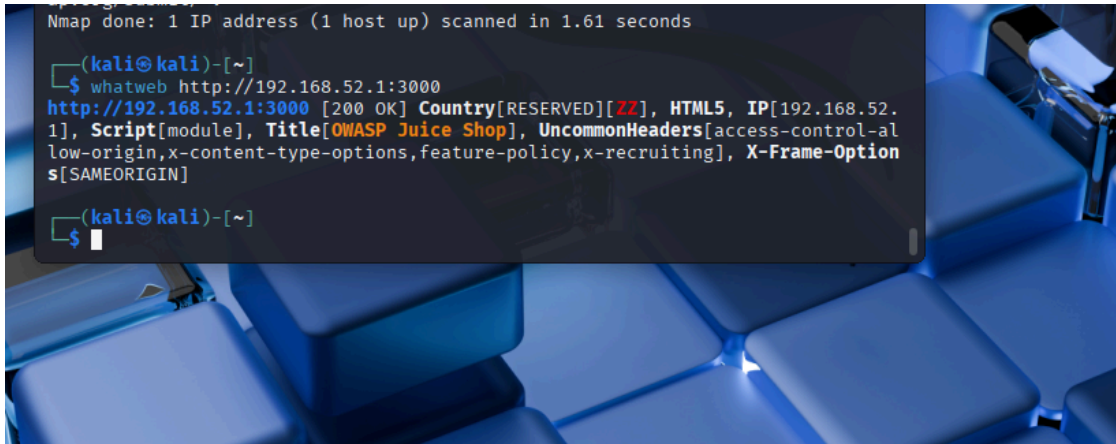


Figure 2 — WhatWeb technology fingerprint

The presence of a wildcard Access-Control-Allow-Origin header was noted during reconnaissance and should be reviewed in a production deployment to ensure cross-origin access is appropriately restricted.

5.3 Robots.txt Analysis

```
curl -i http://192.168.52.1:3000/robots.txt
```

The server returned:

```
User-agent: *
Disallow: /ftp
```

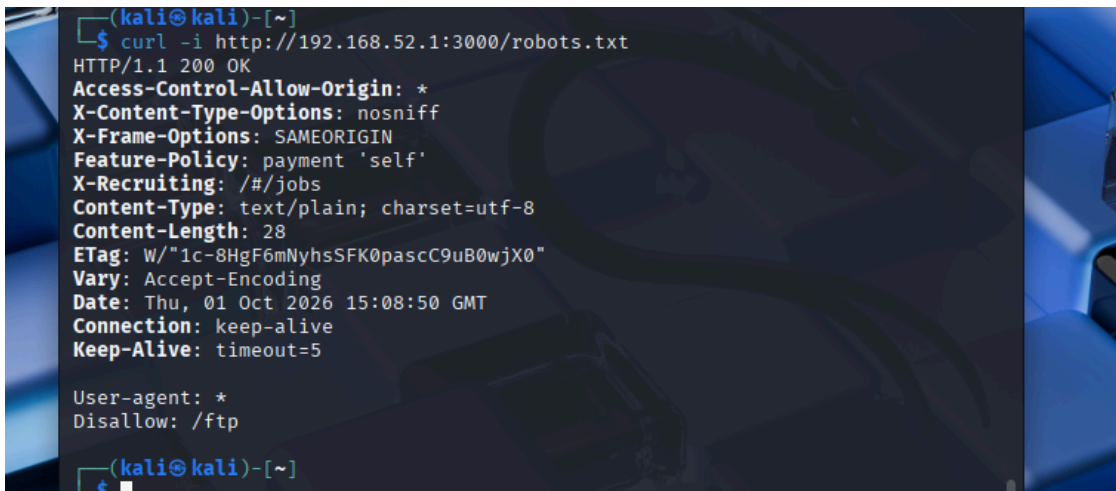


Figure 3 — robots.txt disclosing the /ftp path

robots.txt is not an access-control mechanism. A path listed under Disallow remains directly accessible unless the server separately implements authentication or authorization. In this assessment, the /ftp path could subsequently be accessed directly, confirming this exact weakness.

5.4 Directory Enumeration & Listing Disclosure

Gobuster was used to enumerate common web paths:

```
gobuster dir -u http://192.168.52.1:3000 -w /usr/share/wordlists/dirb/common.txt
```

The enumeration identified the /ftp resource, which returned HTTP 200 OK when requested directly. The directory contents were then listed:

```
curl -s http://192.168.52.1:3000/ftp/ |  
  grep -oE 'href="[^\"]+"' |  
  sed 's/href="//;s/"$//' |  
  sort -u
```

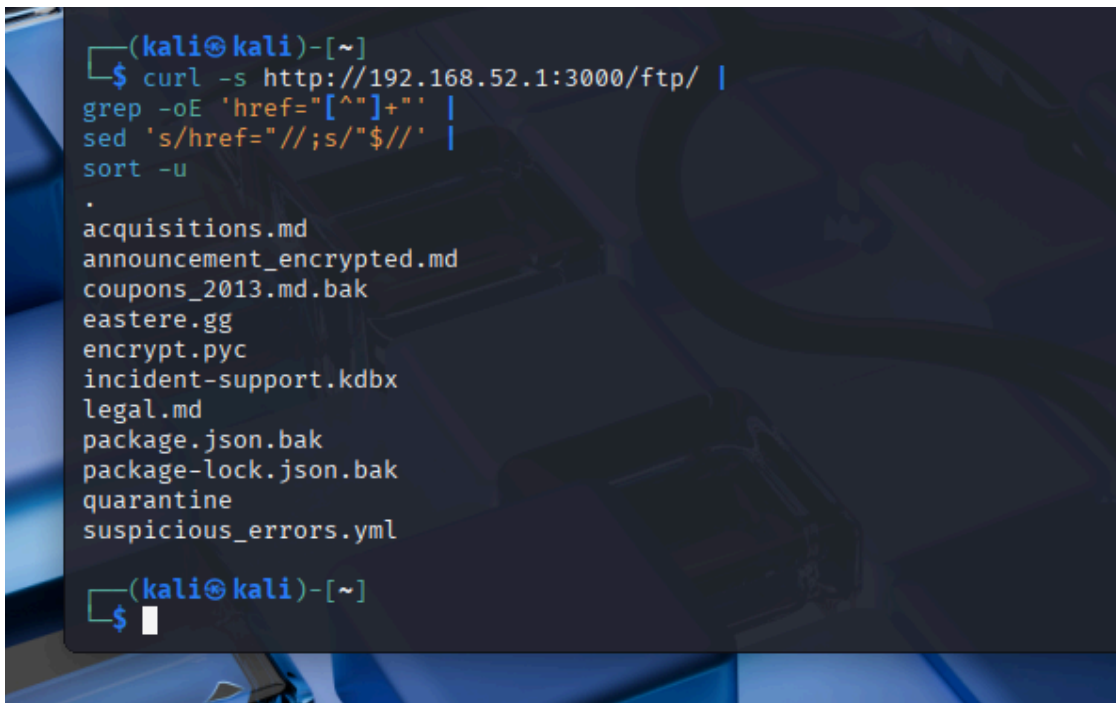


Figure 4 — Directory listing exposed under /ftp/

The following resources were exposed:

```
acquisitions.md  
announcement_encrypted.md  
coupons_2013.md.bak  
eastere.gg  
encrypt.pyc  
incident-support.kdbx  
legal.md  
package.json.bak  
package-lock.json.bak  
quarantine  
suspicious_errors.yml
```

Of particular interest: incident-support.kdbx (a KeePass credential database), package.json.bak / package-lock.json.bak (dependency and build metadata), encrypt.pyc (a compiled Python artifact), and suspicious_errors.yml (a configuration/log-style file).

Confirmed Vulnerability: Directory Listing / Information Disclosure

Attribute	Detail
Severity	Medium
Category	Information Disclosure / Security Misconfiguration
Description	The web server exposed a browsable directory containing internal documents and files. An unauthenticated user could enumerate the full contents of /ftp/.
Evidence	GET /ftp/ returned an HTML directory index listing 11 resources.
Impact	An attacker could obtain information about internal application resources and identify sensitive files for further exploitation.
Juice Shop Validation	Confirmed solved via /api/Challenges — directoryListingChallenge

5.5 Exposed Confidential Documents

acquisitions.md

```
curl -s http://192.168.52.1:3000/ftp/acquisitions.md
```

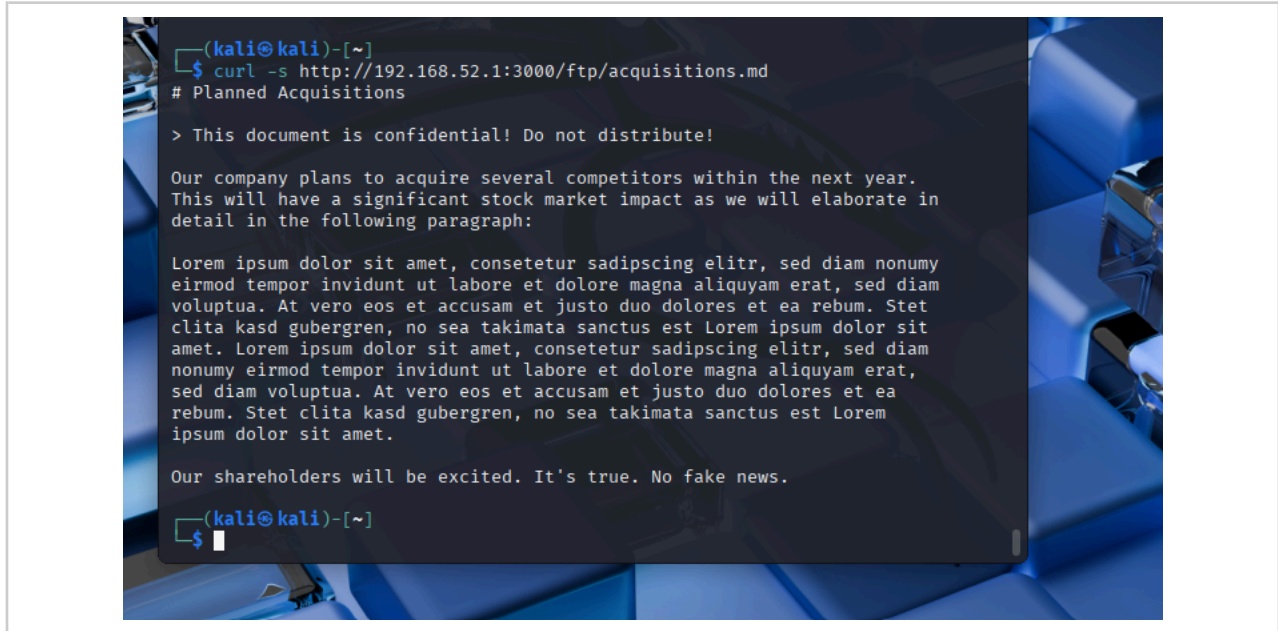
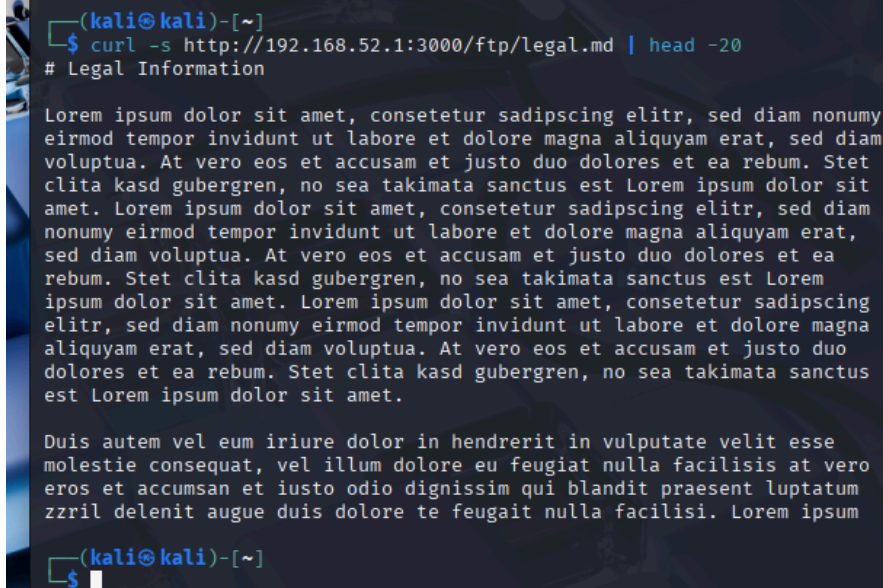


Figure 5 — *acquisitions.md*, self-labeled confidential, served without authentication

The document opens with an explicit confidentiality marking ("This document is confidential! Do not distribute!") describing planned competitor acquisitions, despite being served over a public, unauthenticated path. The body text observed was placeholder (Lorem Ipsum) content, but the exposure pattern itself — a document labeled confidential being openly retrievable — is the security issue, independent of the specific content found in this instance.

legal.md

```
curl -s http://192.168.52.1:3000/ftp/legal.md | head -20
```

A terminal window with a dark background and blue accents. The prompt is (kali@kali)-[~]. The command executed is curl -s http://192.168.52.1:3000/ftp/legal.md | head -20. The output shows the first 20 lines of a file named legal.md, which contains Lorem Ipsum placeholder text. The terminal ends with the prompt (kali@kali)-[~].

```
(kali@kali)-[~]
$ curl -s http://192.168.52.1:3000/ftp/legal.md | head -20
# Legal Information

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy
eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam
voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet
clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit
amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam
nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat,
sed diam voluptua. At vero eos et accusam et justo duo dolores et ea
rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem
ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing
elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna
aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo
dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus
est Lorem ipsum dolor sit amet.

Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse
molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero
eros et accusan et iusto odio dignissim qui blandit praesent luptatum
zzril delenit augue duis dolore te feugait nulla facilisi. Lorem ipsum

(kali@kali)-[~]
$
```

Figure 6 — *legal.md* retrieved without authentication

This file was similarly retrievable directly through the web server with no additional authorization mechanism, confirming the pattern of unauthenticated document access under `/ftp/`.

5.6 File Extension Filtering & Internal Path Disclosure

Several files visible in the directory listing could not be retrieved directly due to a server-side extension restriction:

```
curl -s -i http://192.168.52.1:3000/ftp/suspicious_errors.yml
```

The server responded:

```
403 Error: Only .md and .pdf files are allowed!
```

The response also disclosed an internal application source path:

```
/juice-shop/build/routes/fileServer.js:68:18
```

While the extension filter correctly blocked retrieval of the `.yml` file, the resulting error message leaked the internal filesystem layout and source file responsible for serving `/ftp/` content — a secondary information-disclosure issue. Production applications should avoid exposing internal filesystem paths, source filenames, stack traces, or other framework-specific error details; a generic error response should be returned to the client while detailed debugging data remains in server-side logs.

5.7 Encrypted & Backup File Exposure

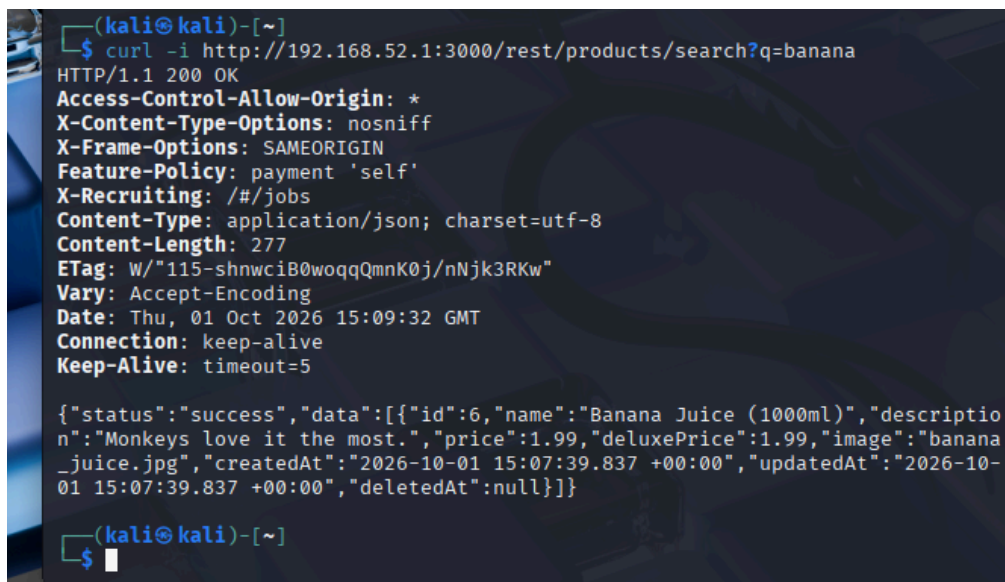
The directory listing also exposed an encrypted document (announcement_encrypted.md) and several backup/internal artifacts: coupons_2013.md.bak, package.json.bak, package-lock.json.bak, encrypt.pyc, and incident-support.kdbx.

- package.json.bak may contain application metadata, dependencies, and build scripts.
- package-lock.json.bak may disclose exact dependency versions, useful for identifying known-vulnerable packages.
- encrypt.pyc is a compiled Python artifact of unknown function, retained in a public path.
- incident-support.kdbx carries the .kdbx extension associated with KeePass password databases — a potentially sensitive credential-management artifact.

Even where direct retrieval of a file is blocked, simply knowing these filenames exist gives an attacker valuable information for follow-on attacks (e.g. targeting known CVEs for the disclosed dependency versions, or attempting offline attacks against the KeePass file if it can later be retrieved by another means).

5.8 Product Search API

```
curl -i "http://192.168.52.1:3000/rest/products/search?q=banana"
```



```
(kali㉿kali)-[~]
$ curl -i http://192.168.52.1:3000/rest/products/search?q=banana
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 277
ETag: W/"115-shnwcIB0woqqQmnK0j/nNjk3RKw"
Vary: Accept-Encoding
Date: Thu, 01 Oct 2026 15:09:32 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success","data":[{"id":6,"name":"Banana Juice (1000ml)","description":"Monkeys love it the most.,"price":1.99,"deluxePrice":1.99,"image":"banana_juice.jpg","createdAt":"2026-10-01 15:07:39.837 +00:00","updatedAt":"2026-10-01 15:07:39.837 +00:00","deletedAt":null}]}

(kali㉿kali)-[~]
$
```

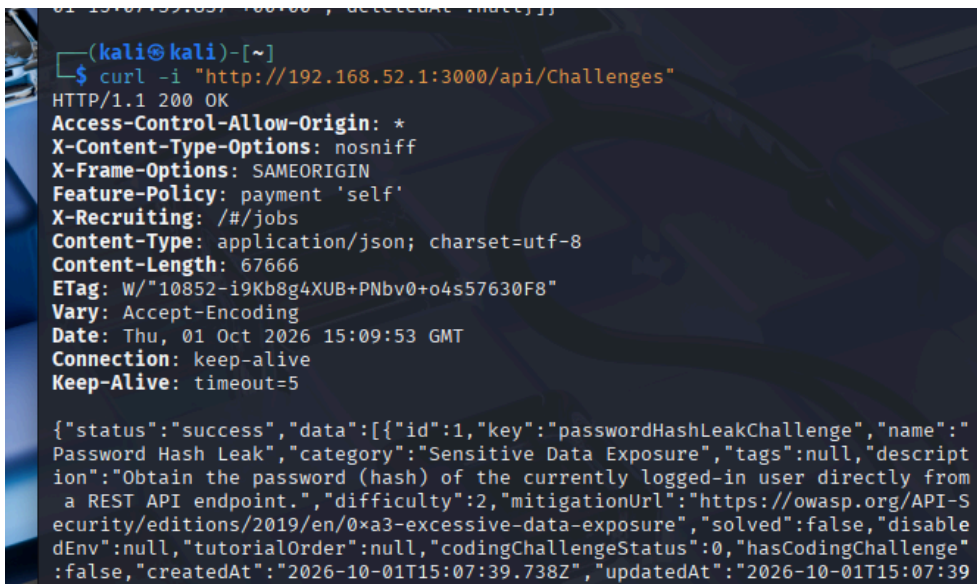
Figure 7 — Product search REST API returning structured JSON

The endpoint accepted a user-controlled query parameter (q) and returned structured JSON product data. A random search term (xyz123) returned an empty result set, confirming the endpoint is live and processes arbitrary input. This endpoint is a natural candidate for injection testing in a follow-up assessment — Juice Shop's own Challenge API separately lists SQL-injection-related challenges (dbSchemaChallenge,

unionSqlInjectionChallenge) associated with search functionality, though exploitation of those was not attempted or confirmed in this phase.

5.9 Challenge API Enumeration

```
curl -i http://192.168.52.1:3000/api/Challenges
```



```
(kali㉿kali)-[~]  
$ curl -i "http://192.168.52.1:3000/api/Challenges"  
HTTP/1.1 200 OK  
Access-Control-Allow-Origin: *  
X-Content-Type-Options: nosniff  
X-Frame-Options: SAMEORIGIN  
Feature-Policy: payment 'self'  
X-Recruiting: /#/jobs  
Content-Type: application/json; charset=utf-8  
Content-Length: 67666  
ETag: W/"10852-i9Kb8g4XUB+PNbv0+o4s57630F8"  
Vary: Accept-Encoding  
Date: Thu, 01 Oct 2026 15:09:53 GMT  
Connection: keep-alive  
Keep-Alive: timeout=5  
  
{  
  "status": "success",  
  "data": [  
    {  
      "id": 1,  
      "key": "passwordHashLeakChallenge",  
      "name": "Password Hash Leak",  
      "category": "Sensitive Data Exposure",  
      "tags": null,  
      "description": "Obtain the password (hash) of the currently logged-in user directly from a REST API endpoint.",  
      "difficulty": 2,  
      "mitigationUrl": "https://owasp.org/API-Security/editions/2019/en/0xa3-excessive-data-exposure",  
      "solved": false,  
      "disableEnv": null,  
      "tutorialOrder": null,  
      "codingChallengeStatus": 0,  
      "hasCodingChallenge": false,  
      "createdAt": "2026-10-01T15:07:39.738Z",  
      "updatedAt": "2026-10-01T15:07:39"
```

Figure 8 — Challenge API response headers (116 challenges, 67,666-byte payload)

```
curl -s "http://192.168.52.1:3000/api/Challenges" | python3 -m json.tool | head -80
```

```
(kali㉿kali)-[~]
$ curl -s "http://192.168.52.1:3000/api/Challenges" |
python3 -m json.tool | head -80
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "key": "passwordHashLeakChallenge",
      "name": "Password Hash Leak",
      "category": "Sensitive Data Exposure",
      "tags": null,
      "description": "Obtain the password (hash) of the currently logged
-in user directly from a REST API endpoint.",
      "difficulty": 2,
      "mitigationUrl": "https://owasp.org/API-Security/editions/2019/en/
0xa3-excessive-data-exposure",
      "solved": false,
      "disabledEnv": null,
      "tutorialOrder": null,
      "codingChallengeStatus": 0,
      "hasCodingChallenge": false,
      "createdAt": "2026-10-01T15:07:39.738Z",
      "updatedAt": "2026-10-01T15:07:39.738Z",
      "ChallengeDependencies": []
    },
    {
```

Figure 9 — Challenge API JSON body, e.g. passwordHashLeakChallenge

```
    },
    {
      "id": 4,
      "key": "registerAdminChallenge",
      "name": "Admin Registration",
      "category": "Improper Input Validation",
      "tags": "With Coding Challenge",
      "description": "Register as a user with administrator privileges."
    },
    {
      "id": 5,
      "key": "adminSectionChallenge",
      "name": "Admin Section",
      "category": "Broken Access Control",
      "difficulty": 3,
      "mitigationUrl": "https://cheatsheetseries.owasp.org/cheatsheets/M
ass_Assignment_Cheat_Sheet.html",
      "solved": false,
      "disabledEnv": null,
      "tutorialOrder": null,
      "codingChallengeStatus": 0,
      "hasCodingChallenge": true,
      "createdAt": "2026-10-01T15:07:39.738Z",
      "updatedAt": "2026-10-01T15:07:39.738Z",
      "ChallengeDependencies": []
    },
    {
      "id": 5,
      "key": "adminSectionChallenge",
      "name": "Admin Section",
      "category": "Broken Access Control",
```

Figure 10 — registerAdminChallenge (Improper Input Validation / Mass Assignment)

The Juice Shop Challenge API reported a total of 116 challenges and was used both to map the application's full built-in attack surface and to confirm which challenges had actually been completed. At the time of testing, directoryListingChallenge was the only challenge marked solved: true. The API nonetheless provides a ready-made map of additional attack surface for prioritized follow-up testing:

Challenge	Security Area
directoryListingChallenge	Directory listing (confirmed solved)
accessLogDisclosureChallenge	Log disclosure
fileWriteChallenge	Arbitrary file modification
dbSchemaChallenge	SQL injection
forgottenDevBackupChallenge / forgottenBackupChallenge	Backup exposure
dlpPastebinDataLeakChallenge	Data leakage
misplacedSignatureFileChallenge	File exposure
retrieveBlueprintChallenge	Sensitive file exposure
ssrfChallenge	Server-side request forgery
unionSqlInjectionChallenge	SQL injection
xxeFileDisclosureChallenge	XXE / file disclosure
exposedMetricsChallenge	Metrics exposure
misplacedIacFiles	Infrastructure-as-code exposure
exposedCredentialsChallenge / leakedApiKeyChallenge	Credential / API key disclosure
iacLeakedKeyChallenge	Infrastructure secret exposure
vulnerableDockerImageChallenge	Container security
registerAdminChallenge	Improper input validation / mass assignment

5.10 Client-Side Resource Analysis

```
curl -s http://192.168.52.1:3000/ | grep -oE 'src="[^"]+\.js[^"]*"'
```

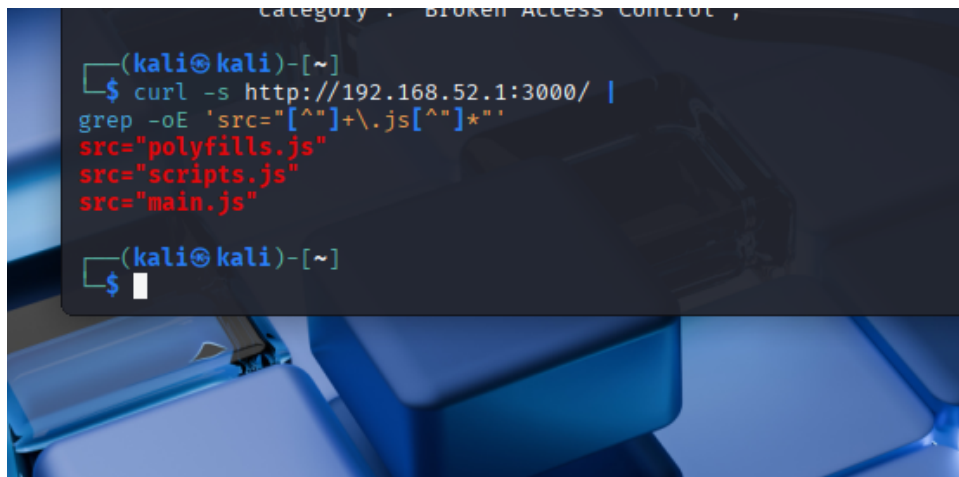


Figure 11 — Client-side JavaScript bundles loaded by the application

The application loads three compiled Angular bundles: `polyfills.js`, `scripts.js`, and `main.js`. These bundles are the natural next step for deeper client-side analysis, as compiled Angular/JavaScript builds routinely retain embedded route tables, API endpoint strings, and configuration values that are recoverable through static analysis of `main.js`.

6. Confirmed Findings Summary

ID	Finding	Evidence	Status
F-01	Directory listing exposed on /ftp/	Directory index returned file listing	Confirmed
F-02	Confidential document publicly accessible	acquisitions.md retrievable, self-labeled confidential	Confirmed
F-03	Internal/backup files exposed via listing	.bak, .pyc, .kdbx, .yaml files visible	Confirmed
F-04	Internal server source path disclosure	fileServer.js:68:18 returned in 403 error	Confirmed
F-05	Public document access without authentication	legal.md directly retrievable	Confirmed
F-06	Encrypted document publicly accessible	announcement_encrypted.md retrievable	Confirmed
F-07	Product search API accessible, unauthenticated	/rest/products/search?q=banana returned data	Confirmed
F-08	Broad additional attack surface identified	Challenge API — 116 challenges enumerated	Identified, not exploited

7. Risk Assessment

The exposure of the /ftp/ directory is the assessment's most significant confirmed issue. In combination, directory indexing, publicly reachable confidential-looking documents, backup artifacts, a credential-database-named file (incident-support.kdbx), encrypted material, and internal source-path disclosure create a meaningful information-disclosure risk.

The assessment did not establish that every exposed file contained exploitable secrets, nor that every Challenge-API item represents a confirmed vulnerability. Impact is therefore described based on confirmed exposure rather than assumed file contents that were not independently verified.

8. Recommendations

8.1 Disable Directory Indexing

Directory listing should be disabled on production web servers so that requesting a directory path returns an appropriate response rather than an enumerable file index.

8.2 Remove Sensitive Files From the Web Root

Files such as *.bak, *.kdbx, *.pyc, *.yaml, and *.json should not be stored in publicly accessible directories unless there is an explicit business requirement paired with appropriate access control.

8.3 Implement Real Authorization

Sensitive documents should require authentication and authorization. Security should never depend on robots.txt or on an attacker not knowing a URL.

8.4 Remove Backup Files From Deployments

Before deploying an application, remove *.bak, *.old, *.backup, *.tmp and other development artifacts from the deployment package.

8.5 Improve Error Handling

The application should not disclose internal paths such as /juice-shop/build/routes/fileServer.js:68:18. Users should receive a generic error message; detailed stack traces and paths should be available only in server-side logs.

8.6 Review CORS Configuration

The observed Access-Control-Allow-Origin: * header should be reviewed and restricted to trusted origins wherever sensitive API functionality is involved.

8.7 Secure Sensitive Application Artifacts

Files such as incident-support.kdbx, package.json.bak, package-lock.json.bak, and encrypt.pyc should be removed from publicly reachable directories. Secrets should be stored in a proper secrets-management system rather than in application source or web directories.

9. Limitations

- The assessment was performed against a lab instance rather than a production system.
- Not all 116 Juice Shop challenges were tested.
- The Challenge API identified numerous possible attack surfaces, but their presence alone does not constitute successful exploitation.
- The contents of some exposed files were not fully analyzed.
- The encrypted announcement file was identified but its encryption mechanism was not established.
- The exposed KeePass database file was identified by filename, but successful extraction or decryption was not established.
- SQL injection challenges were identified but not documented as successfully exploited.
- No claim is made that the application is free from additional vulnerabilities beyond what is documented here.

10. Conclusion

The assessment successfully demonstrated several security weaknesses in the OWASP Juice Shop laboratory application. The most clearly demonstrated issue was directory listing and information disclosure through /ftp/: the directory was accessible despite being disallowed in robots.txt, and it exposed numerous internal resources, including confidential-looking documents, backup artifacts, encrypted material, and a credential-database file.

The application also disclosed an internal server-side source path when an unauthorized file type was requested, and its own Challenge API was used both to validate the directoryListingChallenge as solved and to map 116 total challenges representing further attack surface for prioritized follow-up testing.

This exercise demonstrates the importance of proper access control, disabling directory indexing, removing development and backup artifacts, protecting confidential documents, minimizing error-message disclosure, and reviewing publicly exposed application resources before deployment. It further shows how relatively simple reconnaissance techniques can expose significant application information and reveal additional attack surface for deeper security testing.

Appendix A — Evidence Reproduction Commands

```
# Service identification
nmap -sV -p 3000 192.168.52.1

# Technology fingerprinting
whatweb http://192.168.52.1:3000

# robots.txt
curl -i http://192.168.52.1:3000/robots.txt

# Directory enumeration
gobuster dir -u http://192.168.52.1:3000 -w /usr/share/wordlists/dirb/common.txt

# FTP directory listing
curl -s http://192.168.52.1:3000/ftp/ | grep -oE 'href="[^"]+"' | sed 's/href="//;s/"$//' | sort -u

# Retrieve exposed documents
curl -s http://192.168.52.1:3000/ftp/acquisitions.md
curl -s http://192.168.52.1:3000/ftp/legal.md

# Extension-restriction / error disclosure
curl -s -i http://192.168.52.1:3000/ftp/suspicious_errors.yml

# Product search API
curl -i "http://192.168.52.1:3000/rest/products/search?q=banana"

# Challenge API
curl -s http://192.168.52.1:3000/api/Challenges | python3 -m json.tool

# Client-side JS bundles
curl -s http://192.168.52.1:3000/ | grep -oE 'src="[^"]+\.js[^\"]*"'
```

Appendix B — Assessment Result

Field	Detail
Target	OWASP Juice Shop
IP	192.168.52.1

Field	Detail
Port	3000/TCP
Confirmed challenge	directoryListingChallenge
Primary confirmed issue	Directory Listing / Information Disclosure
Additional confirmed observations	Publicly accessible confidential-looking documents; exposed backup/internal filenames; internal source-path disclosure; publicly accessible encrypted document; accessible application API
Assessment status	Documentation of the performed laboratory assessment completed